UniFlow

AI Research Suggestions

Technical Note · Research directions for the UniFlow MCP / AI layer

UniFlow — AI Research Suggestions

What this is: UniFlow already exposes a multi-tenant business over an MCP tool layer. That makes it an unusually good substrate for **AI research** — you own the tools, the data, and the tenancy, so you can study how AI *selects*, *uses*, and is *governed around* those tools. This document proposes research directions, ranked, with the method and the measurable outcome for each.

---

1. Why this project is good for AI research

- **You control the whole loop** — the 14 tools, the backend, the data. Most students can only prompt a black-box model; you can build, fine-tune, and *evaluate* the tool layer end-to-end.
- **A real, non-trivial task** — mapping a natural-language request to the correct tool + arguments (domain-specialized function calling), in a **multi-tenant, multilingual** setting.
- **A dataset already exists** — 1,293 labelled `(query -> tool call)` examples were generated from the tool catalog, bilingual (EN/FR), with negatives and held-out phrasings.

---

2. Research directions (ranked)

* A. Domain-specialized, fine-tuned tool-selection model *(flagship)*

> **Question:** Can a *small*, fine-tuned open model match — or beat — a *larger* prompted model at picking and filling the correct MCP tool for this domain, entirely on free/local compute?

- **Data:** the 1,293-example dataset (train/val/test with held-out phrasings).
- **Method: QLoRA** (4-bit) fine-tuning of a small open model (Qwen2.5-1.5B / Llama-3.2-3B) on a free Colab/Kaggle T4.

- **Baselines: B1** a larger local model (Llama-3.1-8B via Ollama), zero-shot · **B2** the small model, no fine-tuning · **B3** the fine-tuned small model.
- **Metrics:** tool-selection accuracy, argument F1, full-call accuracy, negative (refuse) accuracy, generalization to held-out phrasings, and **cost/latency**.
- **Why it's strong:** a clean, defensible result with tables and plots; the "specialization beats scale on free compute" angle is genuinely interesting.

B. Agentic Copilot — reasoning + tool orchestration

> Build a **plan -> act -> observe -> respond** agent over the MCP tools (customer assistant + admin copilot).

- **Method:** an explicit agent loop calling the MCP server; swappable "brain" (prompted vs fine-tuned).
- **Eval:** task success rate on scripted scenarios (search -> book, check stock -> restock).
- **Why:** turns "we expose tools" into "we built an agent" — the visible demo *and* an evaluable system.

C. Retrieval-grounded answers (RAG)

> Ground the assistant in the company's own catalog/policies via a vector store so it can't hallucinate prices or availability.

- **Method:** `sentence-transformers` + Chroma/FAISS over products/stores/policies.
- **Eval:** faithfulness / grounding (LLM-as-judge or a manual rubric).

D. Prompt-injection & tool-use safety *(timely, high-novelty)*

> How can a malicious customer prompt misuse the write tools (reserve/pay/cancel), and what defenses work?

- **Method:** build an adversarial prompt set; test attacks; evaluate defenses (allow-lists, confirmation, output constraints).
- **Metrics:** attack success rate before/after defenses.
- **Why:** responsible-AI / security is a hot, publishable-flavored topic and ties directly to your governance layer.

E. Multilingual tool routing (EN / FR / AR)

> Does the router generalize across languages — relevant to an Algerian, multilingual context?

- **Method:** extend the dataset with French/Arabic phrasings; measure per-language accuracy and cross-lingual transfer.
- **Why:** a concrete, under-studied generalization contribution.

F. Demand forecasting / smart restocking *(alternative ML track)*

> Replace the naive "quantity < 10" heuristic with a model that predicts stockouts.

- **Method:** baseline -> ARIMA/Prophet -> small neural net; compare accuracy.
- **Caveat:** needs stock-movement **history**, which the current schema doesn't store — you'd add a movement log and seed synthetic history first.

G. A UniFlow tool-use benchmark *(the evaluation itself as a contribution)*

> Package the dataset + metrics + baselines as a reusable benchmark for business-domain tool-calling.

- **Why:** benchmarks are a legitimate research output; it frames all the above under one measurable umbrella.

---

3. Recommended combination

For a strong PFE with both an engineering system and a research result:

- **Research core: A (fine-tuned tool-selection)** + **G (benchmark)** — the novel, measurable contribution with tables/plots.
- **Engineering demo: B (agent)** + **C (RAG)** — the visible, working Copilot.
- **Optional standout chapter: D (safety)** or **E (multilingual)** if time allows.

This is the "Ambitious" scope: a governed, specialized, agentic AI layer, *evaluated*.

---

4. What you need (all free / local)

Need	Choice
Dataset	[ok] Already built (1,293 examples, bilingual)
Baseline / larger	Ollama (Llama-3.1-8B / Qwen2.5-7B), local

Need	Choice
model	
Fine-tuning compute	Free Colab / Kaggle T4 GPU (QLoRA on a 1–3B model)
Fine-tuning stack	`transformers` + `peft` + `trl` + `bitsandbytes`
RAG	`sentence-transformers` + Chroma/FAISS
Evaluation	a small Python harness + `matplotlib`

---

5. Deliverables for the report

- **Results table:** B1 vs B2 vs B3 across all metrics.
- **Plots:** per-tool accuracy bars, a tool-selection confusion matrix, and a cost/latency-vs-accuracy scatter.
- **The benchmark** (dataset + harness) as a reusable artifact.
- **A short discussion:** where small-specialized wins, where it fails, and the safety/multilingual findings.

---

6. One-line summary

> UniFlow's MCP layer is the perfect testbed for a focused research question — **"can a small, fine-tuned model out-route a bigger one on this domain, safely and multilingually, on free compute?"** — with a dataset already in hand and a clear path to measurable results.